pySeqAlign

Relational sequence alignment, multiple alignment, and logos

A. Karwath

github.com/athro/pySeqAlign · PyPI: `pyseqalignment`

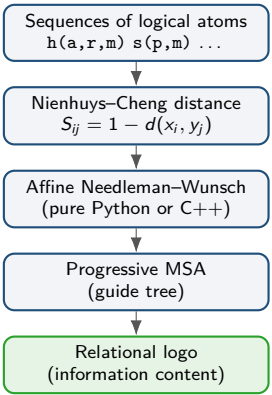
Aligning sequences of structured logical atoms — the ILP-2006 layer:
Karwath & Kersting, *Relational Sequence Alignments and Logos* (ILP 2006).

Outline

What pySeqAlign does

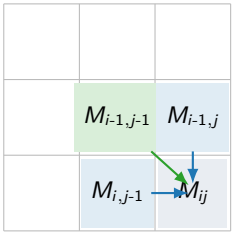
A domain-independent sequence-alignment library that works on **structured** elements (logical atoms), not just flat symbols.

- **Alignment**: Needleman–Wunsch (linear & affine gaps), Smith–Waterman (k-best local).
- **Relational distance**: Nienhuys–Cheng distance over logical atoms $\Rightarrow$ similarity $S = 1 - d$.
- **Substitution matrices**: BLOSUM / PAM bundled.
- **Multiple alignment**: progressive MSA over a neighbour-joining guide tree.
- **Relational logos**: information-content logos of aligned atoms (+ per-column least-general generalisation).
- **ILP backends**: Aleph / Popper (optional, SWI-Prolog).
- **Optional C++ aligner**: pybind11 affine kernel,



Global alignment: Needleman–Wunsch

Fill a DP matrix; each cell takes the best of three moves (match/mismatch, gap in one sequence, gap in the other):



$$M_{ij} = \max \begin{cases} M_{i\text{-}1,j\text{-}1} + S_{ij} & \text{(diag: match)} \\ M_{i,j\text{-}1} + w & \text{(gap in seq. 1)} \\ M_{i\text{-}1,j} + w & \text{(gap in seq. 2)} \end{cases}$$

- **Affine gaps**: separate open/extend costs (3-matrix $M/I_x/I_y$ recurrence) – `NeedlemanWunschAffine`.
- **Local**: Smith–Waterman with k-best non-overlapping hits.
- The score S_{ij} is *pluggable* – this is the hook for relational distances.

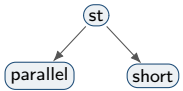
From flat symbols to logical atoms

Real sequences carry internal structure. An SSE or a tagged word is an atom, not a letter:

`he(h(right,alpha),long)` `st(parallel,short)` `nn(balloon)` `vbd(blew)`

The **Nienhuys–Cheng** distance treats a ground atom as a hierarchy: same functor $\Rightarrow$ distance shrinks with the fraction of matching arguments (at most 0.5); different functor $\Rightarrow$ distance 1.

$$d(p(t_1,\ldots,t_n),p(s_1,\ldots,s_n)) = \tfrac{1}{2n}\sum_{i=1}^{n} d(t_i,s_i), \qquad d(p,q)=1 \text{ if } p\neq q.$$



`st(parallel,short)`

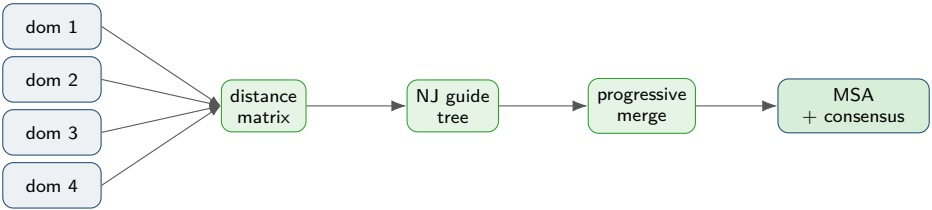
Used as alignment score: $S_{ij} = 1 - d(x_i, y_j)$.

Example:
$d($`st(parallel,short)`$,$`st(parallel,long)`$) = \tfrac{1}{4}(0+1) = 0.25 \Rightarrow S = 0.75$.

Implemented in `pyseqalign.scoring.distance.AtomDistance`.

Multiple sequence alignment

Progressive MSA: all-pairs distances → neighbour-joining guide tree → merge along the tree.
Parameterised by any scoring function.



Why "any scoring function" matters

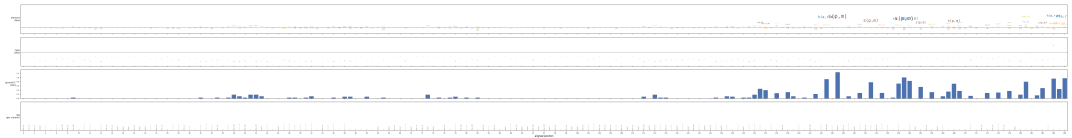
The same MSA accepts the fixed Nienhuys–Cheng distance *or* a reward matrix learned by pyREAL's boosting – one MSA engine, two regimes.

Relational sequence logos (ILP 2006)

A logo turns an alignment into one picture. Per aligned column i, the information content (Gorodkin / KL form)

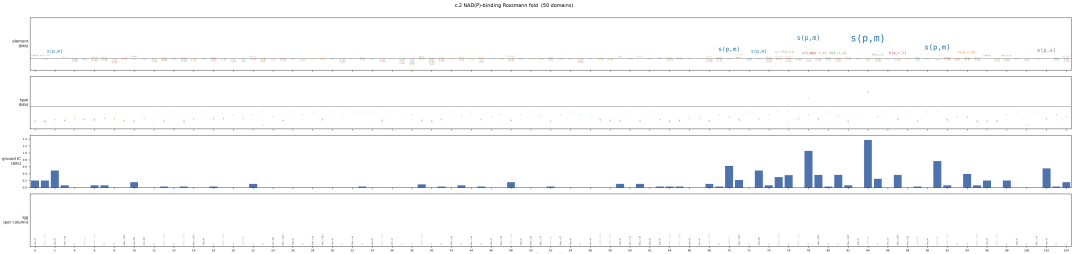
$$I_i = \sum_k q_{ik} \log_2 \frac{q_{ik}}{p_k}$$

sets the stack height; each atom's glyph height is its share $q_{ik} \log_2(q_{ik}/p_k)$ (rarer-than-expected atoms drawn upside-down). pySeqAlign adds a per-column **least-general generalisation** track (the conserved relational pattern).



SCOP fold **c.1 (TIM β/α-barrel)** – the conserved alternating parallel strands `s(p,m)` and α-helices, reconstructed by `examples/scop_logos.py`.

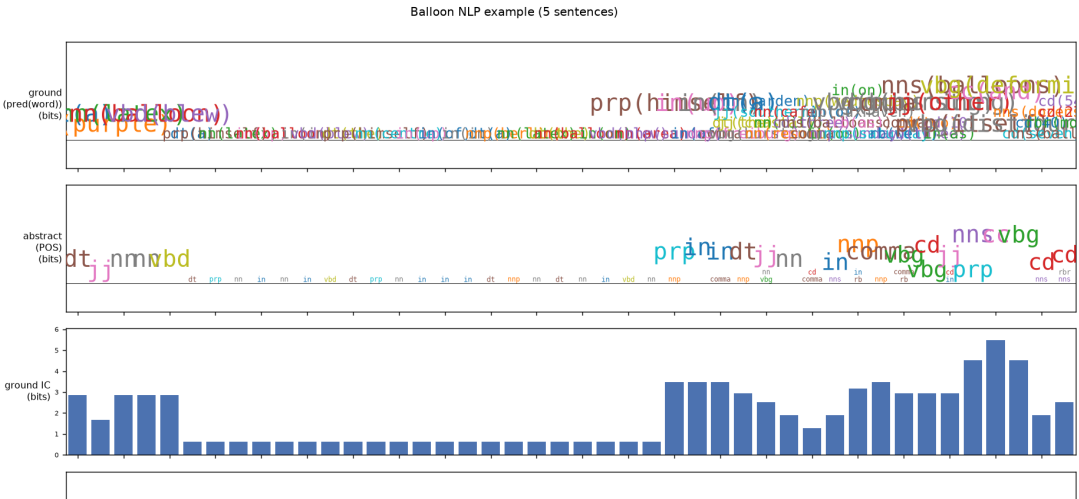
Logos match the SCOP expert descriptions



SCOP fold **c.2 (NAD(P)-binding Rossmann fold)**: the information peaks fall on the conserved parallel β-strands `s(p,m)` – exactly the "6 parallel strands" the SCOP database describes (paper Fig. 6). No learning involved – pure alignment + IC.

The running example: balloon (NLP)

Five POS-tagged sentences (`pred(word)` atoms) aligned and rendered as a logo: the shared syntactic skeleton `dt nn nn vbd prp in in dt ...` emerges at the abstract (part-of-speech) level.



Putting it together

```
from pyseqalign.msa import progressive_msa
from pyseqalign.logo import relational_logo
from pyseqalign.scoring.distance import AtomDistance

store = {1:('h','a','r','m'), 2:('s','p','m'), 3:('h','a','r','l')}
seqs  = {'d1':[1,2,3], 'd2':[1,3,2], 'd3':[2,3]}

sc  = AtomDistance(atom_store=store, gap_score=-0.5)  # NC distance
msa = progressive_msa(seqs, sc, gap_open=-1.0, gap_extend=-0.1)
relational_logo(list(msa.aligned_sequences.values()),
                store, 'logo.png', title='fold c.1')
```

`pip install pyseqalignment` – pure-Python core; the C++ aligner is compiled automatically at install when a compiler is present, else it falls back silently.

Summary

- pySeqAlign aligns sequences of **structured logical atoms** with classic DP algorithms and an ILP relational distance.
- **MSA + relational logos** reproduce the ILP-2006 results standalone – SCOP fold logos match the expert descriptions; the balloon logo reproduces the paper's information content.
- The scoring function is pluggable: the same engine serves the fixed Nienhuys–Cheng distance and **pyREAL's learned rewards**.
- Pure-Python core; optional auto-compiled C++ accelerator.

pyREAL adds the learning → see the companion deck.